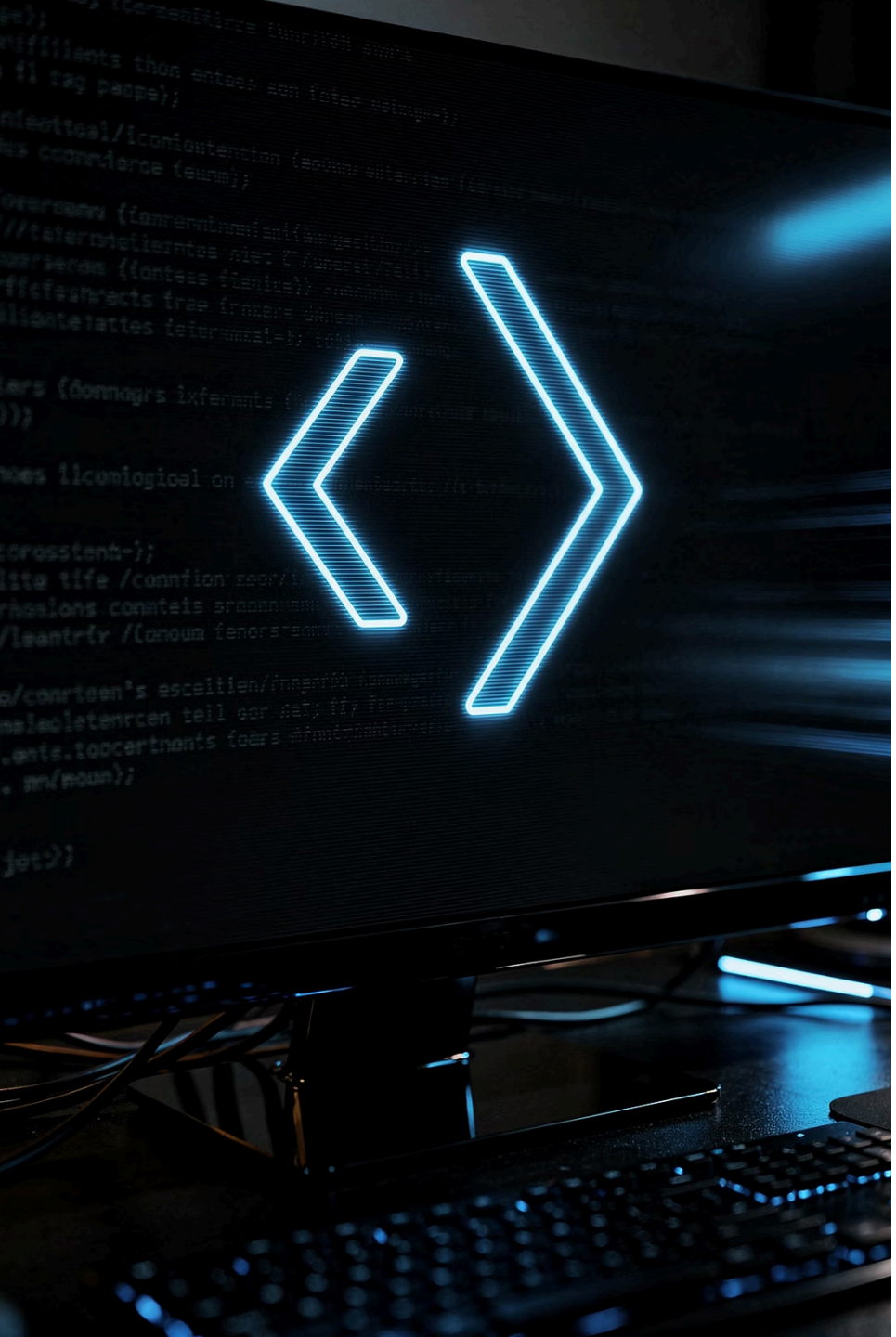


Module 4: Directives and Pipes

Enhancing UI Behavior and Data Presentation

ANGULAR FUNDAMENTALS

MODULE 4

Module Overview

By the end of this module, you will be able to:

01

Understand Directives

Grasp what Angular directives are and how they extend HTML behavior

02

Differentiate Directive Types

Distinguish between component, attribute, and structural directives

03

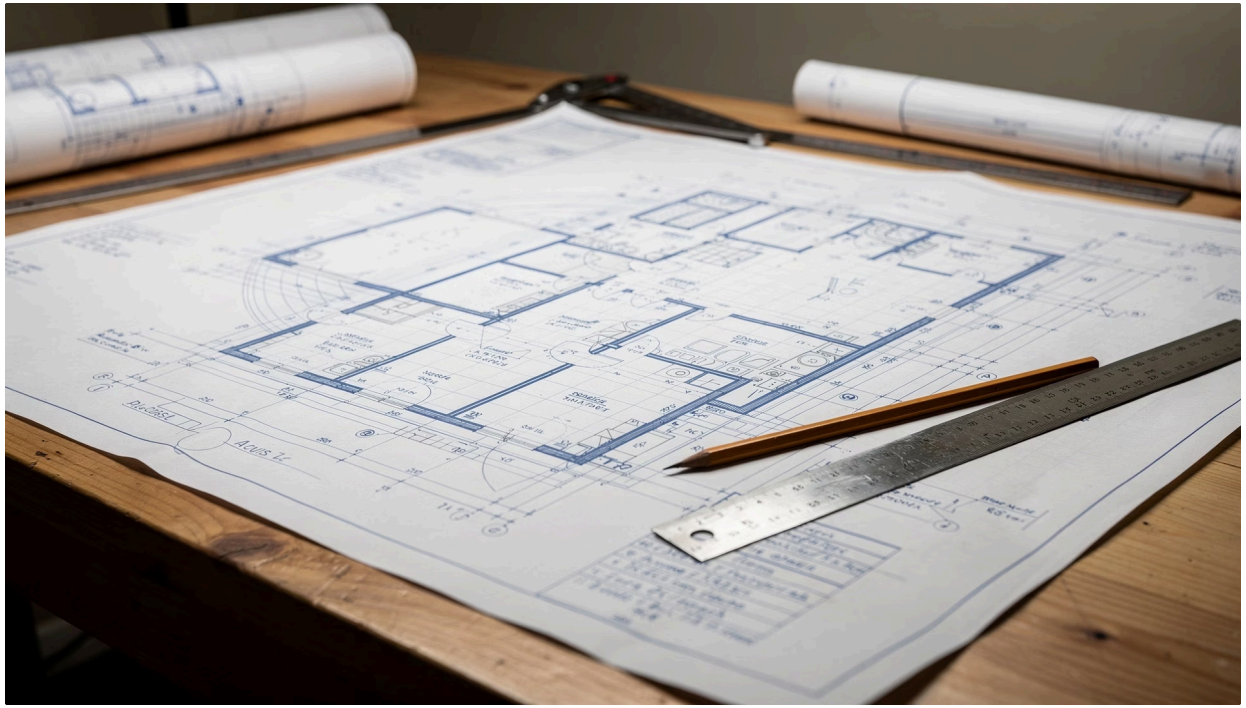
Build Custom Directives & Pipes

Create reusable UI behavior and custom data transformations

04

Apply Best Practices

Use pipes efficiently within Angular's change detection cycle



CORE CONCEPT

What Are Directives?

A **Directive** is a class that extends or modifies the behavior and appearance of HTML elements — without creating a new component.

Key Point: Components are directives with a template. Directives add behavior to existing elements.

Reuse UI Behavior

Write once, apply anywhere across your templates

Reduce Duplication

Eliminate repeated logic scattered across components

Improve Maintainability

Centralized behavior is easier to test and update

Types of Directives

Angular provides three categories of directives, each serving a distinct purpose in the UI layer.

Component

Creates a self-contained UI view with its own template, styles, and logic. The most common directive type.

Attribute Directive

Changes the **appearance or behavior** of an existing element, component, or another directive.

Structural Directive

Reshapes the DOM by **adding, removing, or replacing** elements based on conditions or data.

→ Highlight invalid form fields · Disable buttons dynamically · Show or hide sections · Render lists from data

Built-in Directives

Attribute Directives

ngClass

Dynamically add or remove CSS classes

ngStyle

Apply inline styles conditionally

Structural Directives (Control Flow)

@if

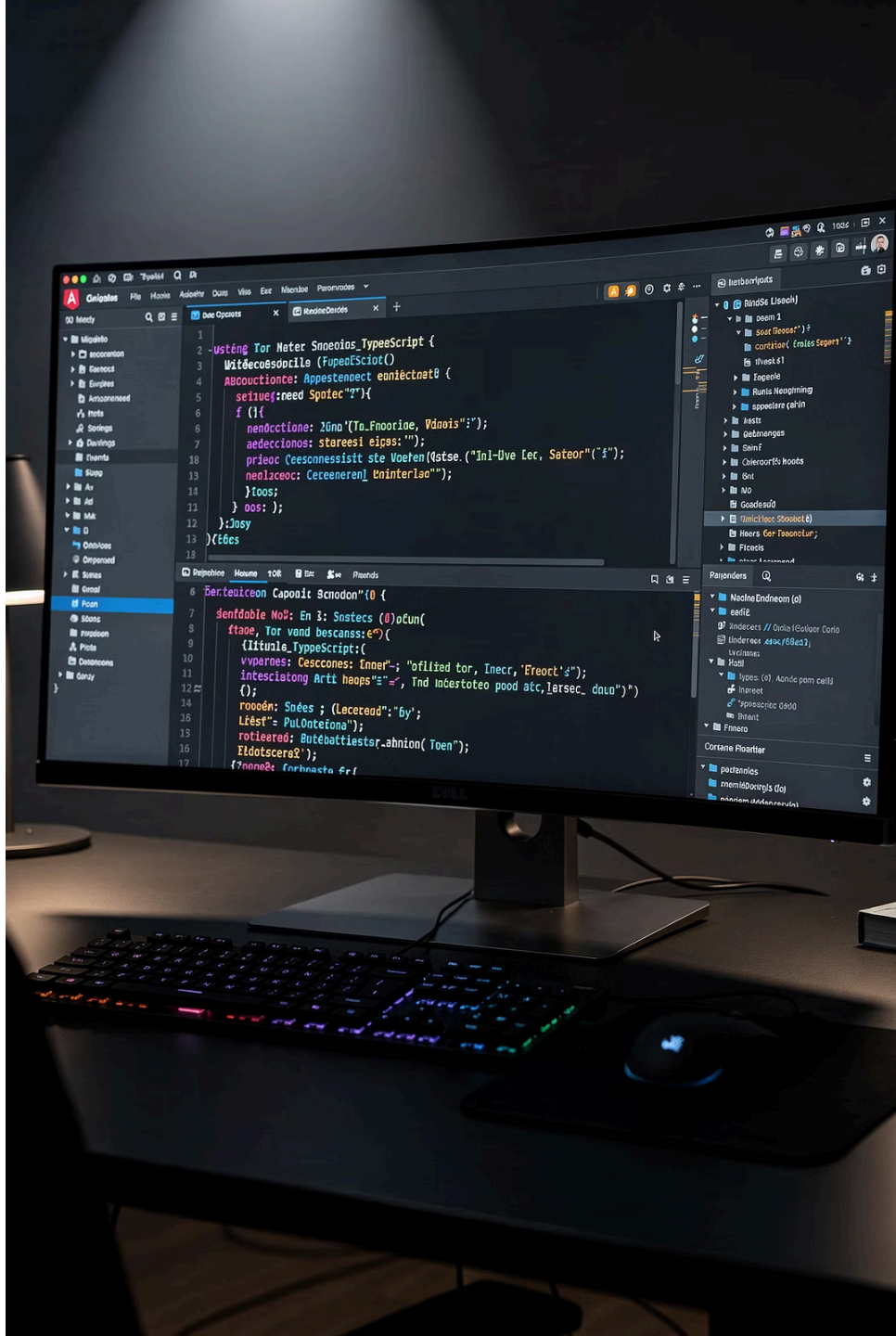
Conditional rendering

@for

Iterate over collections

@switch

Multi-case rendering



Custom Attribute Directives

Custom directives let you encapsulate reusable UI behavior and apply it declaratively across your templates — keeping your components clean and focused.

Benefits

- **Reusable logic** — apply the same behavior to any element
- **Consistent UX** — uniform behavior across the entire app
- **Cleaner components** — move UI concerns out of component classes



Highlight Invalid Fields

Visually flag form inputs that fail validation



Auto-Focus Inputs

Automatically focus a control on render



Permission-Based UI

Show or disable elements based on user roles

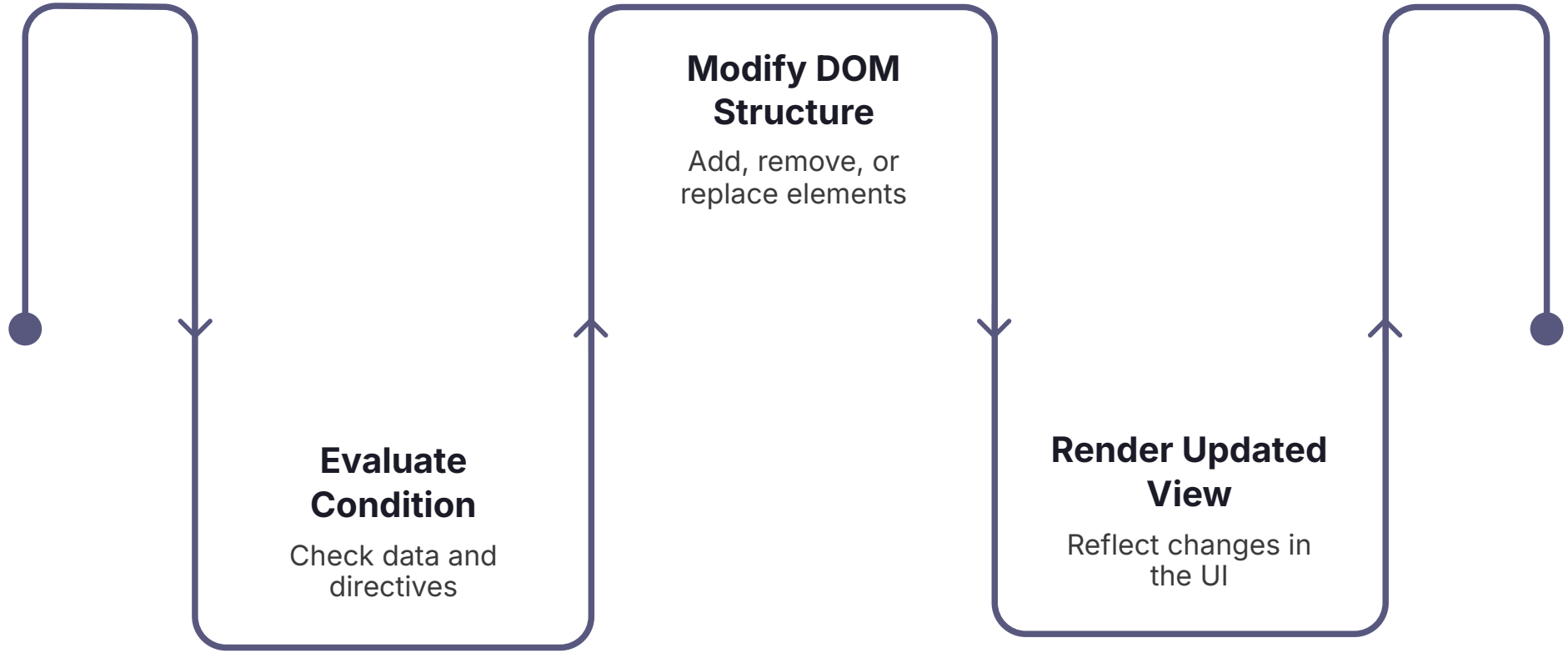


Custom Tooltips

Rich, reusable tooltip behavior on any element

Structural Directives

Structural directives reshape the DOM itself — adding, removing, or replacing elements to create dynamic, data-driven UIs.



Common scenarios include rendering employee records with **@for**, showing loading spinners with **@if**, building dynamic menus with **@switch**, and displaying empty-state messages when collections are empty.



CORE CONCEPT

What Are Pipes?

A **Pipe** transforms data before displaying it in the UI — without ever mutating the original value. Think of it as a formatting layer between your data and your template.

Key Point: Pipes improve presentation while keeping your business data completely unchanged.

Date Formatting

Display raw timestamps as readable dates

Currency & Numbers

Format monetary values and decimals

Text Transforms

Uppercase, lowercase, title case

JSON Display

Pretty-print objects for debugging

Built-in Pipes

Angular ships with a comprehensive set of pipes for the most common formatting needs.

Pipe	Purpose	Example Use
`DatePipe`	Format dates	Report dates
`CurrencyPipe`	Format currency	Employee salary
`DecimalPipe`	Format numbers	Metrics & scores
`PercentPipe`	Display percentages	Order completion
`UpperCasePipe`	Text to uppercase	Status labels
`TitleCasePipe`	Capitalize words	User names
`JsonPipe`	Display objects	Debug output

Enterprise Examples

Invoice Amount

```
{{ invoice.total | currency:'USD' }}
```

Hire Date

```
{{ employee.hireDate | date:'mediumDate' }}
```

Completion Rate

```
{{ order.progress | percent }}
```

Custom Pipes

When built-in pipes don't cover your needs, custom pipes encapsulate reusable, testable data transformation logic directly in your templates.

Employee ID Formatter

Convert raw IDs to readable codes like `EMP-00142`

Number Masking

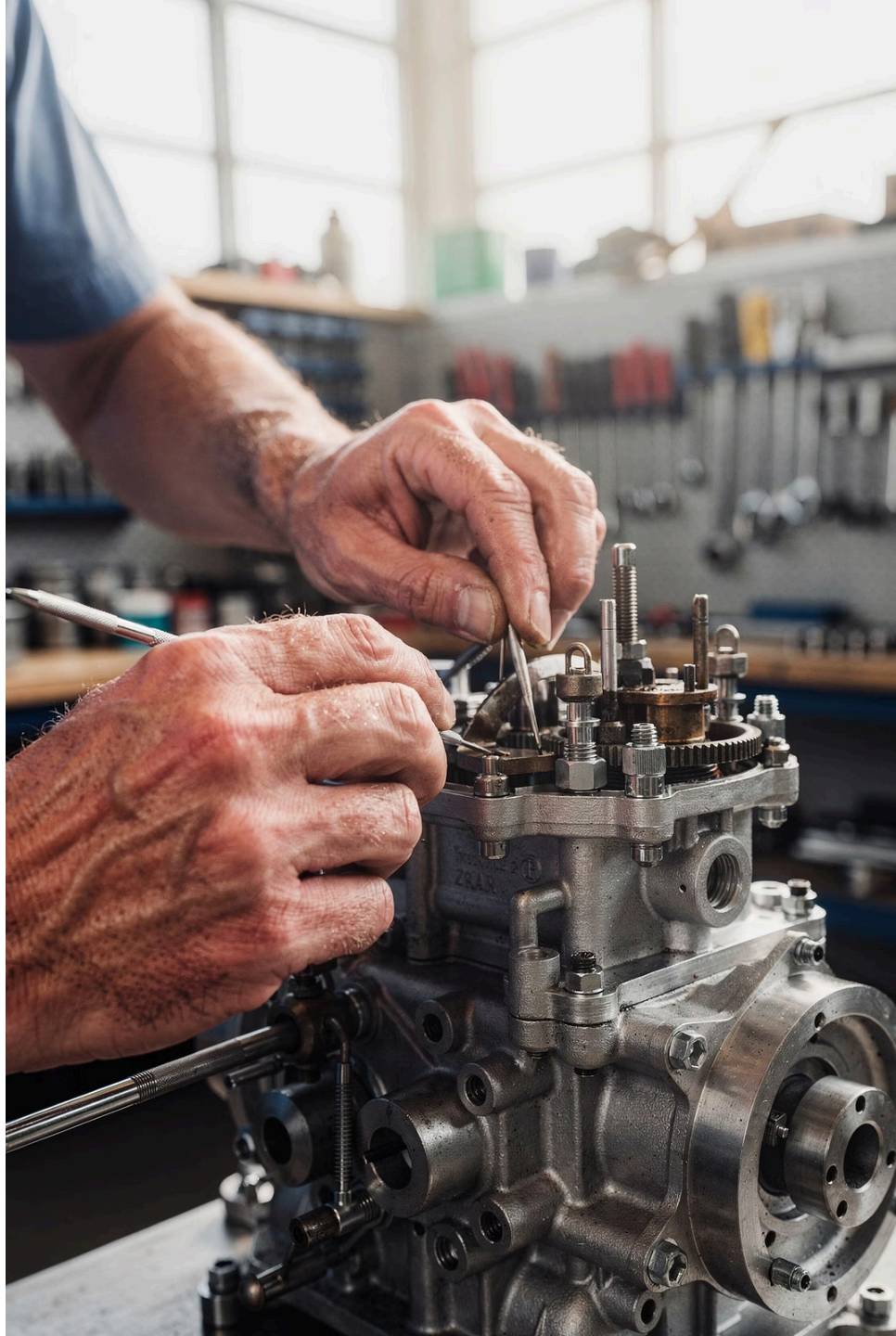
Mask mobile numbers and credit cards: `••••-1234`

Status Formatter

Map enum values to human-readable labels

File Size & Duration

Display bytes as `MB` or seconds as `hh:mm:ss`



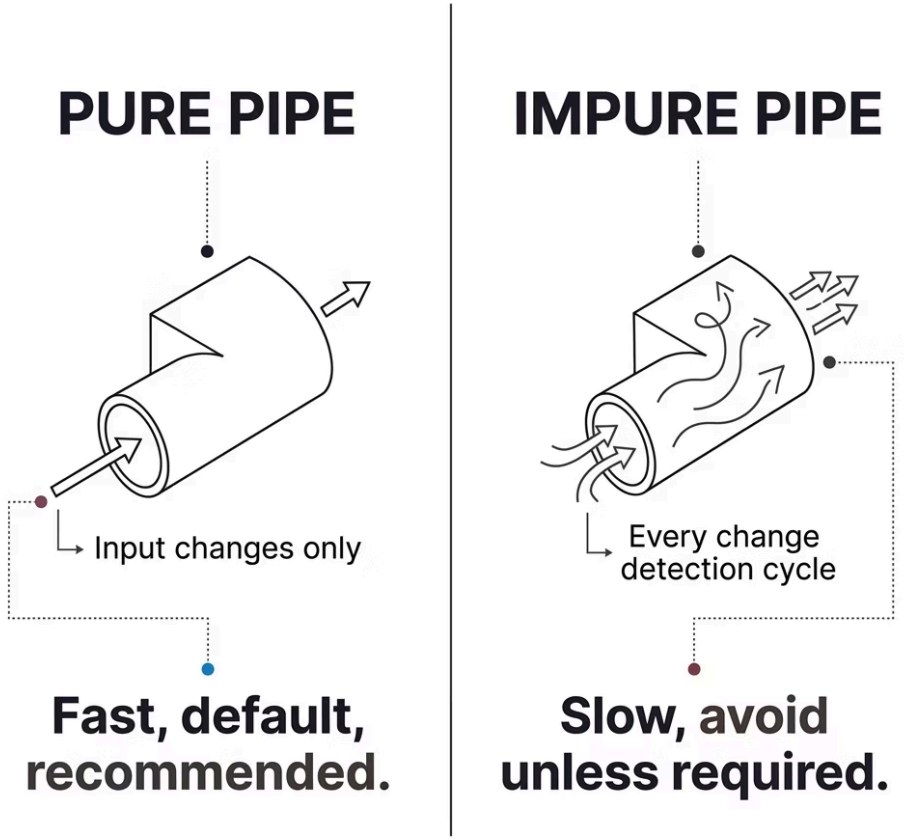
Pipe Performance Considerations

Pipes execute during Angular's **change detection cycle**. A poorly written pipe can silently degrade your app's performance.

Best Practices

- **Prefer pure pipes** — they run only when inputs change
- **Avoid heavy computations** — keep logic lightweight
- **Cache expensive results** — memoize if recalculation is costly
- **Delegate to services** — use services for complex business logic

> **Recommendation:** Pipes should *format* data, not execute business operations.



Directives vs. Pipes

Both are powerful Angular tools — but they solve different problems. Choosing the right one keeps your architecture clean.

Directives

- Modify **UI behavior** and appearance
- Work directly with the **DOM**
- Add interaction and visual changes
- Can change structure or styling

Pipes

- Transform **displayed data** values
- Work with **values**, not elements
- Improve presentation layer
- Cannot modify the DOM



Directive → Changes HTML Behavior



Pipe → Changes Data Presentation

Module Summary

What We Covered

- Angular Directives — concept and purpose
- Component, Attribute, and Structural types
- Built-in directives: `ngClass`, `ngStyle`, `@if`, `@for`
- Custom attribute directives for reusable UI behavior
- Built-in and custom pipes for data formatting
- Pipe performance best practices (pure vs. impure)

Up Next

Module 5: Signals & Angular Reactivity

- Signals & Computed Signals
- Effects
- Signal Inputs & Outputs